Number Theory I

Divisibility, gcd/lcm, primes, sieves & factorization

Learning goals

- Divisibility vocabulary and the **fundamental theorem of arithmetic**
- Enumerate divisors and test primality in **O(√n)**
- **Euclid's algorithm** for gcd; lcm without overflow
- **Sieve of Eratosthenes** in O(n log log n)
- **Smallest-prime-factor sieve** → factorize any $x \le 10^7$ in O(log x)
- Divisor count / sum formulas from a factorization
- Harmonic-series sieves: O(n log n) "for each d, for each multiple"

Motivation: CSES 1713 — Counting Divisors

$n \le 10^5$ queries, each $x \le 10^6$: print $d(x)$, the number of divisors of x.

```
3
16  → 5   (1 2 4 8 16)
17  → 2
18  → 6   (1 2 3 6 9 18)
```

- Trial division per query: $10^5 \cdot \sqrt{10^6} = 10^8$ — borderline
- Precompute $d(x)$ for **all** $x \le 10^6$ once: $\sim 1.4 \cdot 10^7$ steps — easy

Divisibility basics

- $a \mid b$ ("a divides b") $\iff b = a \cdot k$ for some integer k
- If $a \mid b$ and $a \mid c$ then $a \mid (b \cdot x + c \cdot y)$ for all integers x, y
- **Prime**: $p > 1$ whose only divisors are 1 and p
- **Fundamental theorem**: every $n > 1$ is uniquely $n = p_1^{e_1} \cdot p_2^{e_2} \cdot \ldots \cdot p_k^{e_k}$

```
360 = 2³ · 3² · 5
```

- Divisors of $n \leftrightarrow$ choices of exponents $0 \le f_i \le e_i$

Divisors come in pairs

If d | n then (n/d) | n, and one of d, n/d is ≤ √n.

```
vector<long long> divs;
for (long long d = 1; d * d <= n; d++)
    if (n % d == 0) {
        divs.push_back(d);
        if (d != n / d) divs.push_back(n / d);   // avoid double √n
    }
```

- **O(√n)**: n = 10^{12} → 10^{6} iterations
- Use `d * d <= n`, not `d <= sqrt(n)` (floating-point error)
- Max divisor count: 1344 for n ≤ 10^{9}, 6720 for n ≤ 10^{12}

Primality test by trial division

```
bool isPrime(long long n) {
    if (n < 2) return false;
    for (long long d = 2; d * d <= n; d++)
        if (n % d == 0) return false;
    return true;
}
```

- O(√n) per number
- Fine for a few numbers up to ~10^{14}
- Many queries or a range → **sieve**
- Huge n (10^{18}) → Miller–Rabin (later in the course)

Factorization by trial division

```
vector<pair<long long,int>> factorize(long long n) {
    vector<pair<long long,int>> f;
    for (long long p = 2; p * p <= n; p++) {
        if (n % p) continue;
        int e = 0;
        while (n % p == 0) { n /= p; e++; }
        f.push_back({p, e});
    }
    if (n > 1) f.push_back({n, 1});   // leftover prime > √(original n)
    return f;
}
```

Only primes ever divide here: composites' factors were removed earlier.

$600851475143 = 71 \cdot 839 \cdot 1471 \cdot 6857$

gcd and Euclid's algorithm

Key fact: gcd(a, b) = gcd(b, a mod b), and gcd(a, 0) = a.

```
gcd(84, 36) → gcd(36, 12) → gcd(12, 0) = 12
```

```cpp
long long g(long long a, long long b) { return b ? g(b, a % b) : a; }
// C++17: std::gcd(a, b) in <numeric>
```

- **O(log min(a, b))** — every two steps the larger value at least halves
- Worst case: consecutive Fibonacci numbers

lcm and gcd properties

- gcd(a, b) · lcm(a, b) = a · b
- `lcm = a / gcd(a, b) * b` — **divide first** to avoid overflow
- `std::lcm` exists, but overflow is your problem
- gcd(a, b, c) = gcd(gcd(a, b), c) — fold over an array
- gcd(a, b) = gcd(a, b − a) → gcd of differences:
  $\gcd(a_1 + x, \ldots, a_n + x) = \gcd(a_1 + x, a_2 - a_1, \ldots, a_n - a_1)$
- In terms of factorizations: gcd takes **min** exponents, lcm takes **max**

Sieve of Eratosthenes

Cross out multiples of each prime, starting from p^2.

```
2  3  4  5  6  7  8  9  10 11 12 13 14 15 16 17 18 19 20
   p  x  p  x  p  x  x  x  p  x  p  x  x  x  p  x  p  x
```

```cpp
vector<bool> isPrime(n + 1, true);
isPrime[0] = isPrime[1] = false;
for (int i = 2; (long long)i * i <= n; i++)
    if (isPrime[i])
        for (int j = i * i; j <= n; j += i) isPrime[j] = false;
```

Why start at p^2, and why $O(n \log \log n)$?

- Any composite $j < p^2$ with factor p has a smaller prime factor → already crossed out
- Work = Σ over primes $p \le n$ of $n/p = n \cdot \Sigma\, 1/p \approx \mathbf{n \ln \ln n}$
- $n = 10^7$: ~$3 \cdot 10^7$ operations, `vector<bool>` uses 1.25 MB
- $n = 10^8$ is still possible with `vector<bool>` / bitset (~1 s)

Smallest prime factor (SPF) sieve

```
vector<int> spf(n + 1, 0);
for (int i = 2; i <= n; i++)
    if (spf[i] == 0)                          // i is prime
        for (int j = i; j <= n; j += i)
            if (spf[j] == 0) spf[j] = i;
```

Factorize any x ≤ n by repeatedly dividing by `spf[x]`:

```
84: spf=2 → 42: spf=2 → 21: spf=3 → 7: spf=7 → 1
84 = 2² · 3 · 7
```

O(log x) per number (each step at least halves x).

Linear sieve (optional)

Each composite is crossed out exactly once, by its smallest prime factor:

```
vector<int> lp(n + 1, 0), primes;
for (int i = 2; i <= n; i++) {
    if (lp[i] == 0) { lp[i] = i; primes.push_back(i); }
    for (int p : primes) {
        if (p > lp[i] || (long long)i * p > n) break;
        lp[i * p] = p;
    }
}
```

O(n), gives SPF + list of primes. In practice similar speed to Eratosthenes.

Counting and summing divisors

$n = p_1^{e_1} \cdot \ldots \cdot p_k^{e_k}$

Quantity	Formula	$360 = 2^3 \cdot 3^2 \cdot 5$
number of divisors $d(n)$	$\prod (e_i + 1)$	$4 \cdot 3 \cdot 2 = 24$
sum of divisors $\sigma(n)$	$\prod (p_i^{(e_i+1)} - 1)/(p_i - 1)$	$15 \cdot 13 \cdot 6 = 1170$
number of distinct primes $\omega(n)$	k	3

Each divisor = an independent choice of exponent for each prime.

Harmonic sieve: all divisor counts

```
vector<int> d(M + 1, 0);
for (int i = 1; i <= M; i++)
    for (int j = i; j <= M; j += i)   // i divides j
        d[j]++;
```

Work = M/1 + M/2 + … + M/M = M · H_M ≈ **M ln M**

M = 10^6 → ~1.4 · 10^7 steps. Solves CSES 1713 instantly.

Same pattern: sum of divisors, "for every value, count multiples present"

→ **CSES 1081 Common Divisors**.

Worked example: CSES 1081 — Common Divisors

Array of n ≤ $2 \cdot 10^5$ values ≤ 10^6; max gcd over all pairs.

- cnt[v] = how many times v occurs
- For each candidate g from 10^6 down to 1:
  multiples = cnt[g] + cnt[2g] + cnt[3g] + …
- First g with multiples ≥ 2 is the answer

O(M log M) — never touches the n^2 pairs.

Primes: useful facts

- $\pi(n)$, the number of primes $\le n$, $\approx n / \ln n$
  ($\pi(10^6) = 78{,}498$, $\pi(10^7) = 664{,}579$)
- Gaps between consecutive primes below 10^9 are < 300
  → "next prime ≥ x" by trial division is fast
- Every prime > 3 is $6k \pm 1$
- n has at most one prime factor $> \sqrt{n}$
- $2 \cdot 3 \cdot 5 \cdot 7 \cdot 11 \cdot 13 \cdot 17 \cdot 19 \cdot 23 \cdot 29 > 10^9$ → ≤ 9 distinct primes for $n \le 10^9$

Segmented sieve (idea)

Primes in [L, R] with $R \le 10^{12}$, $R - L \le 10^6$:

1. Sieve primes up to $\sqrt{R} \approx 10^6$
2. For each such p, cross out multiples of p in [L, R], starting at $\max(p^2, \lceil L/p \rceil \cdot p)$
3. Remaining numbers (≥ 2) are prime

$O((R - L) \log \log R + \sqrt{R})$. Classic: SPOJ PRIME1.

Choosing a method

Situation	Method
one n ≤ 10^{14}, test / factor	trial division O(√n)
many queries, values ≤ 10^{7}	SPF sieve, O(log x) per query
all primes ≤ 10^{8}	Eratosthenes with `vector<bool>`
property of every x ≤ 10^{6}	harmonic sieve O(M log M)
primes in a window of a huge range	segmented sieve

Pitfalls

- `d <= sqrt(n)` with doubles → off-by-one for perfect squares
- `i * i` overflowing `int` in the sieve loop — cast to `long long`
- Forgetting the leftover prime `n > 1` after trial division
- Counting $\sqrt{n}$ twice when enumerating divisor pairs
- `lcm = a * b / gcd` overflows; divide first
- 1 is **not** prime; 0 and 1 need special handling
- SPF array of 10^7 ints = 40 MB — check the memory limit

Practice problems

- CSES 1713 — Counting Divisors
- CSES 1081 — Common Divisors
- Codeforces 26A — Almost Prime
- Codeforces 230B — T-primes
- Codeforces 776B — Sherlock and his girlfriend
- Codeforces 1350A — Orac and Factors
- Codeforces 1047C — Enlarge GCD
- AtCoder ABC 177 E — Coprime
- SPOJ PRIME1 — Prime Generator (segmented sieve)

Summary

- Divisors pair up around √n → O(√n) enumeration, test, factorization
- gcd via Euclid in O(log); lcm = a / gcd · b
- Eratosthenes O(n log log n); SPF sieve → O(log x) factorization
- d(n) = Π(e_i + 1); σ(n) = Π (p^(e+1) − 1)/(p − 1)
- "For each d, for each multiple" costs O(M log M)
- Next week: **modular arithmetic**, fast power, inverses, nCr mod p